

Exercise 4: Constrained Refactoring (Live) — 25 minutes

Scenario

`InventoryService.java` validates a reorder level the same way in two places. Look at `validateForCreate` and `validateForUpdate`:

```
private static InventoryItem validateForCreate(InventoryItem item) {
    InventoryItem normalized = normalize(item);
    validateTextFields(normalized);
    if (normalized.quantity() < 0) {
        throw invalid("Quantity cannot be negative");
    }
    if (normalized.reorderLevel() < 0) {
        throw invalid("Reorder level cannot be negative");
    }
    return normalized.withId(0);
}

private static InventoryItem validateForUpdate(InventoryItem item) {
    InventoryItem normalized = normalize(item);
    validateTextFields(normalized);
    if (normalized.reorderLevel() < 0) {
        throw invalid("Reorder level cannot be negative");
    }
    return normalized;
}
```

The `if (normalized.reorderLevel() < 0) { throw invalid(...) }` block is duplicated verbatim. Refactor it into a single private helper while preserving behavior exactly.

Read the two methods carefully before you start: they are *not* symmetric, and the asymmetry is not part of what you were asked to fix.

Explicit invariants

Give attendees this checklist before they start:

- Public APIs must not change (`InventoryService` 's public methods: `findAll` , `findById` , `create` , `update` , `delete` , `isHealthy` keep their exact signatures).
- No behavior changes — the same inputs must throw the same `InventoryException` with the same message, for both create and update.
- **`validateForUpdate` must keep accepting a negative `quantity`** . Only `validateForCreate` checks it today. Tidying that difference away is a behavior change, however reasonable it looks — and no existing test will catch it. If you think the update path *should* check quantity, that's a separate change with its own test, not something to slip into a refactor.
- Existing tests must continue to pass, in particular `InventoryServerTest.java` .
- No new dependencies.
- Only `src/main/java/com/atlas/inventory/InventoryService.java` may be changed.
- The final diff should be smaller than approximately 30 lines.

Prompt template

```
refactor the duplicated reorder-level validation in validateForCreate and
validateForUpdate in InventoryService.java.
```

```
constraints:
```

- preserve all behavior, including the exact exception type and message.
- do not change public methods, names, or signatures.
- do not change any other validation rule (quantity, text fields).
- do not modify tests unless a test is required to preserve an existing behavior.
- change only `src/main/java/com/atlas/inventory/InventoryService.java`.
- keep the diff as small as possible.

```
before editing, identify the behavior that must remain unchanged.
```

What you should expect

- A new private helper, for example `requireNonNegativeReorderLevel(InventoryItem item)` , called from both `validateForCreate` and `validateForUpdate` .
- No change to `normalize` , `validateTextFields` , `translateConstraintViolation` , or any public method signature.

- `mvn test -Dtest=InventoryServerTest` still passes unchanged.

Run the full test suite to confirm nothing broke — not a named subset, since by this point the suite may also contain the test classes you added in exercises 2 and 3:

```
mvn test
```

Review activity

Compare, side by side:

- the original and final diff (`git diff`),
- the existing tests (do they still make sense, and did any need touching?),
- the agent's explanation of the change,
- their own understanding of the code before the refactor.

Then consider:

"Would you approve this pull request without reading the diff?"

It's best to be on the cautious side — even a constrained, small, behavior-preserving refactor should be read line by line before merging, because "no behavior change" is a claim the agent makes, not a guarantee.

Trainer notes

This exercise's real payload is the asymmetry between the two methods: `validateForCreate` rejects a negative `quantity`, `validateForUpdate` does not.

An agent told to "remove the duplication" will quite often unify the two methods into one shared validator — which silently adds a quantity check to the update path. **Every existing test still passes**, because nothing covers updating an item to a negative quantity. That is the whole lesson in one diff: a green suite is evidence about the tests you have, not about the change you made.

Let attendees hit it if they do; if nobody does, demonstrate it yourself by prompting without the quantity invariant in the constraint list and comparing the two diffs. Background on why the gap exists is in [TRAINER-NOTES.md](#).